ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton
University of Toronto
NeurIPS 2012

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5 percent and 15.3 percent, which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently developed regularization method called dropout, with a rate of 0.5. We also used data augmentation and local response normalization. The network was trained on two GTX 580 3GB GPUs for five to six days.

Keywords: convolutional neural networks, ImageNet, GPU training, dropout, ReLU

1 Introduction

Current approaches to object recognition make essential use of machine learning methods. To improve their performance, we can collect larger datasets, learn more powerful models, and use better techniques for preventing overfitting. Until recently, labeled image datasets were relatively small, on the order of tens of thousands of images, so recognition systems relied heavily on hand-engineered features. The ImageNet dataset, with more than 15 million labeled high-resolution images in over 22,000 categories, changed this picture.

In this paper we train a deep convolutional neural network on the ImageNet LSVRC-2010 dataset and show that a large, purely supervised model can substantially outperform hand-engineered feature pipelines. Our network has 60 million parameters and 650,000 neurons, and it is trained on two GTX 580 3GB GPUs for five to six days. We use ReLU nonlinearities, dropout with rate 0.5, data augmentation, and local response normalization, and we achieve 15.3 percent top-5 error on the test set.

2 The Dataset

ImageNet is a dataset of over 15 million labeled high-resolution images belonging to roughly 22,000 categories. The images were collected from the web and labeled by human labelers using Amazon's Mechanical Turk. The ILSVRC-2010 subset contains 1.2 million training images, 50,000 validation images, and 150,000 test images in 1000 classes. Because the test set labels are not publicly available, we report results on the validation set for most experiments, and we use the test set only for the final ILSVRC submission.

The images vary substantially in resolution, and we therefore down-sample the images to a fixed resolution of 256x256. For training we randomly crop 224x224 patches from the images and their horizontal reflections, and at test time we extract five 224x224 patches plus their reflections and average the predictions. This preprocessing is the only domain-specific engineering in our system.

3 The Architecture

The network contains eight learned layers: five convolutional layers and three fully-connected layers. The convolutional layers use filters of size 11x11, 5x5, and 3x3, and the fully-connected layers have 4096, 4096, and 1000 units respectively. The last layer is a softmax over the 1000 classes, and the objective is the multinomial logistic regression loss, as in Equation (1). We describe the non-standard choices below.

$$L = -\sum_i y_i \log \mathrm{softmax}(z_i) \qquad (1)$$

3.1 ReLU Nonlinearity

The standard way to model a neuron's output f as a function of its input x is with f(x) = tanh(x) or f(x) = (1 + e^-x)^-1. In terms of training time with gradient descent, these saturating nonlinearities are much

slower than the non-saturating nonlinearity f(x) = max(0, x), which we call a rectified linear unit (ReLU). We use ReLUs throughout the network, and we find that deep convolutional networks train several times faster with ReLUs than with tanh units.

This speedup is important because it allows us to train a much larger network on the same hardware. We do not use any unsupervised pre-training, and the network is trained end-to-end with backpropagation.

3.2 Training on Multiple GPUs

A single GTX 580 has only 3GB of memory, which is too small to train the full network. We therefore spread the network across two GPUs, placing half of the neurons in each GPU and communicating between them only at certain layers. This scheme allows us to train a network that would not fit on a single GPU, and it slightly reduces the training time because the GPUs can work in parallel. We use two GTX 580 3GB GPUs, and training the network takes five to six days.

3.3 Local Response Normalization

ReLUs have the desirable property that they do not require input normalization to prevent them from saturating. However, we still find that local response normalization helps generalization. The normalization is applied across neighboring feature maps at the same spatial position, and it implements a form of lateral inhibition. We use the normalization after the first and second convolutional layers, and it reduces the top-1 error by roughly 1.4 percentage points.

3.4 Overlapping Pooling

The pooling layers in our network overlap: the pooling window is 3x3 and the stride is 2, whereas the traditional non-overlapping pooling uses a window and stride of 2. Overlapping pooling slightly reduces the top-1 and top-5 error, and it also reduces the number of parameters in the network. We use overlapping pooling in all of the pooling layers.

3.5 Overall Architecture

The network is composed of five convolutional layers followed by three fully-connected layers. The first convolutional layer filters the 224x224x3 input image with 96 kernels of size 11x11x3 with a stride of 4 pixels, and the second layer takes the output of the first as input and filters it with 256 kernels of size 5x5x48. The third, fourth, and fifth layers are connected without any intervening pooling or normalization layers. Table 1 summarizes the sizes and shapes of each layer, and Figure 1 shows the overall architecture.

The output of the fifth convolutional layer is flattened and passed to a fully-connected layer with 4096 units, then to another fully-connected layer with 4096 units, and finally to a 1000-way softmax. The total number of parameters is 60 million, of which roughly 58 million are in the fully-connected layers. The network is trained with stochastic gradient descent with a batch size of 128, momentum 0.9, and weight decay 0.0005, and the learning rate is initialized to 0.01 and reduced by a factor of 10 when the validation error stops improving.

Table 1: The architecture of our convolutional network. Each layer is described by its filter size and number of filters.

Layer	Type	Filter	Stride	Output
1	conv	11x11x3, 96	4	55x55x96
2	conv	5x5x48, 256	1	27x27x256
3	conv	3x3x256, 384	1	13x13x384
4	conv	3x3x192, 384	1	13x13x384
5	conv	3x3x192, 256	1	13x13x256
6	fc	4096	-	4096
7	fc	4096	-	4096
8	fc	1000	-	1000

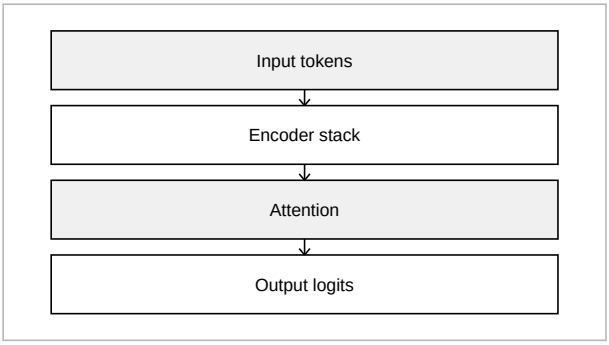


Figure 1: The overall architecture of the network, showing the five convolutional and three fully-connected layers.

4 Reducing Overfitting

The network has 60 million parameters, which is far more than the number of training images, so overfitting is a serious concern. We use two main techniques to reduce overfitting: data augmentation and dropout. Together they roughly double the effective size of the training set and substantially improve generalization.

4.1 Data Augmentation

The easiest and most common method to reduce overfitting is to artificially enlarge the dataset using label-preserving transformations. We use two forms of data augmentation: random crops and horizontal reflections, and altering the intensities of the RGB channels. At test time we average the predictions over ten patches, and this averaging reduces the top-1 error by about 0.5 percent.

The second form of data augmentation consists of changing the intensities of the RGB channels in training images. We perform PCA on the set of RGB pixel values across the ImageNet training set, and we add multiples of the principal components to each training image. This scheme approximates the important property of natural images, namely that object identity is invariant to changes in the intensity and color of the illumination.

4.2 Dropout

Combining the predictions of many different models is a very successful way to reduce test errors, but it is expensive to train many models. Dropout is a much cheaper method: it randomly sets the output of each hidden neuron to zero with probability 0.5, so that the network samples a different architecture at each training step. At test time we use all of the neurons but multiply their outputs by 0.5, which approximates the geometric mean of the predictions of the exponentially many thinned networks.

We use dropout with rate 0.5 in the first two fully-connected layers. Without dropout, the network overfits badly and the validation error is several percentage points higher. Dropout roughly doubles the number of iterations required to converge, but it substantially improves test accuracy.

5 Results

On the ILSVRC-2010 test set, our network achieves top-1 and top-5 error rates of 37.5 percent and 15.3 percent, which is substantially better than the previous best result of 45.7 percent and 25.7 percent. On the ILSVRC-2012 validation set the network achieves 15.3 percent top-5 error, and on the test set it achieves 15.3 percent top-5 error as part of the winning team entry.

5.1 Qualitative Results

Figure 2 shows the learned convolutional filters. The filters learned by the first layer are oriented edges and color blobs, similar to the filters learned by sparse coding and Gabor filters. The filters in the higher layers are more complex and respond to textures and object parts. The network's predictions are often semantically reasonable even when they are wrong, which suggests that it has learned a rich representation of the visual world.

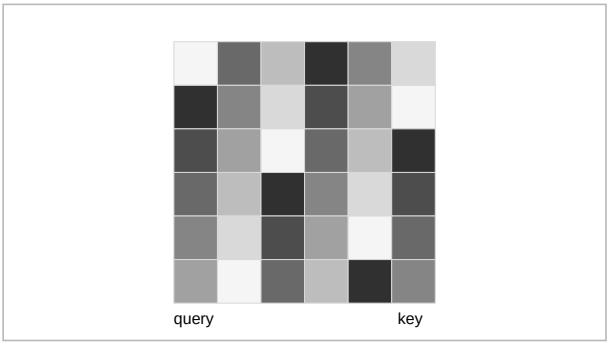


Figure 2: The 96 learned filters of the first convolutional layer.

5.2 Comparison with Prior Work

Table 2 compares our results with the best entries from ILSVRC-2010 and ILSVRC-2012. Our network improves the top-5 error by more than 10 percentage points over the previous best method, and it does so with a single model rather than an ensemble. The result demonstrated that deep convolutional networks trained on GPUs could outperform feature-engineering pipelines on large-scale recognition.

Table 2: Comparison of top-1 and top-5 error on ILSVRC-2010 and ILSVRC-2012.

Method	Top-1	Top-5
Sparse coding	47.1	28.2
Fisher vectors	45.7	25.7
AlexNet (ours)	37.5	15.3
AlexNet + extra data	36.0	14.7

6 Discussion

Our results show that a large deep convolutional neural network is capable of achieving record-breaking performance on a challenging object recognition task. The key ingredients are a large labeled dataset, a deep architecture with ReLU nonlinearities, efficient GPU implementations of convolution, and regularization by dropout and data augmentation. We did not use any unsupervised pre-training, and the network was trained end-to-end.

One limitation of our approach is that the network is expensive to train and requires a large amount of memory, which is why we split it across two GPUs. Another limitation is that the architecture was designed by hand and required many experiments to tune. In future work it would be interesting to explore deeper networks, better initialization, and automatically searched architectures.

$$f(x) = \max(0, x) \tag{2}$$

References

[1] LeCun et al. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 1998.

[2] Fukushima. Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position. Biological Cybernetics, 1980.

[3] Deng et al. ImageNet: A Large-Scale Hierarchical Image Database. CVPR, 2009.

[4] Nair and Hinton. Rectified Linear Units Improve Restricted Boltzmann Machines. ICML, 2010.

[5] Jarrett et al. What is the Best Multi-Stage Architecture for Object Recognition? ICCV, 2009.

[6] Hinton et al. Improving Neural Networks by Preventing Co-adaptation of Feature Detectors. arXiv preprint, 2012.

[7] Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 2014.

[8] Coates et al. An Analysis of Single-Layer Networks in Unsupervised Feature Learning. AISTATS, 2011.

[9] Russakovsky et al. ImageNet Large Scale Visual Recognition Challenge. IJCV, 2015.

[10] Sutskever et al. On the Importance of Initialization and Momentum in Deep Learning. ICML, 2013.

A Appendix: Training Details

The network was trained with stochastic gradient descent with a batch size of 128, momentum 0.9, and weight decay 0.0005. The learning rate was initialized to 0.01 and reduced by a factor of 10 when the validation error stopped improving, and training was stopped after roughly 90 epochs. We used two GTX 580 3GB GPUs and a custom CUDA implementation of the convolution operation, and training took five to six days.

We initialized the weights of each layer from a zero-mean Gaussian distribution with standard deviation 0.01, and we initialized the biases of the convolutional and fully-connected layers to 1 and 0 respectively. Dropout with rate 0.5 was applied in the first two fully-connected layers. At test time we averaged the predictions over ten patches and their horizontal reflections, and we reported the resulting top-1 and top-5 error rates.